

Complejidad computacional

Programación II

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

En caso de que resuelva el problema, también tenemos que pensar en el **costo** que se tiene que pagar para que el algoritmo funcione.

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

En caso de que resuelva el problema, también tenemos que pensar en el **costo** que se tiene que pagar para que el algoritmo funcione.

Este costo es la cantidad de recursos del que necesita disponer el algoritmo para poder trabajar:

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

En caso de que resuelva el problema, también tenemos que pensar en el **costo** que se tiene que pagar para que el algoritmo funcione.

Este costo es la cantidad de recursos del que necesita disponer el algoritmo para poder trabajar:

- Memoria necesaria
- Tiempo de ejecución

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

En caso de que resuelva el problema, también tenemos que pensar en el **costo** que se tiene que pagar para que el algoritmo funcione.

Este costo es la cantidad de recursos del que necesita disponer el algoritmo para poder trabajar:

- Memoria necesaria → **Complejidad espacial**
- Tiempo de ejecución

¿Qué es la complejidad de un algoritmo?

Cuando creamos un algoritmo, lo primero que pensamos es si resolverá el problema planteado o no.

En caso de que resuelva el problema, también tenemos que pensar en el **costo** que se tiene que pagar para que el algoritmo funcione.

Este costo es la cantidad de recursos del que necesita disponer el algoritmo para poder trabajar:

- Memoria necesaria → **Complejidad espacial**
- Tiempo de ejecución → **Complejidad temporal**

Complejidad espacial

Cantidad de memoria que un algoritmo consume o utiliza durante su ejecución.

No desarrollamos este tema en la materia.

Complejidad temporal

Tiempo que necesita un algoritmo para ejecutarse.

Complejidad temporal

Tiempo que necesita un algoritmo para ejecutarse.

Nos centraremos en la **complejidad temporal** por ser el recurso más crítico.

Pero...

Complejidad temporal

Tiempo que necesita un algoritmo para ejecutarse.

Nos centraremos en la **complejidad temporal** por ser el recurso más crítico.

Pero...

¿Cómo hacemos para medir el tiempo?

Complejidad temporal

Tiempo que necesita un algoritmo para ejecutarse.

Nos centraremos en la **complejidad temporal** por ser el recurso más crítico.

Pero...

¿Cómo hacemos para medir el tiempo?

Un mismo algoritmo puede tardar horas en una PC de pocos recursos o algo antigua y segundos en otra PC que tenga mejores procesador y memoria.

Entonces,

Complejidad temporal

Tiempo que necesita un algoritmo para ejecutarse.

Nos centraremos en la **complejidad temporal** por ser el recurso más crítico.

Pero...

¿Cómo hacemos para medir el tiempo?

Un mismo algoritmo puede tardar horas en una PC de pocos recursos o algo antigua y segundos en otra PC que tenga mejores procesador y memoria.

Entonces, en lugar de contar tiempo, contaremos **instrucciones**.

Conteo de instrucciones/operaciones

Consideraremos que las siguientes instrucciones atómicas consumen un ciclo del procesador. y las usaremos como unidad en el conteo de instrucciones/operaciones:

Conteo de instrucciones/operaciones

Consideraremos que las siguientes instrucciones atómicas consumen un ciclo del procesador. y las usaremos como unidad en el conteo de instrucciones/operaciones:

- operadores aritméticos: **+**, **-**, *****, **/**, **%**, **++**, **--**,
- operaciones lógicas: **&&**, **||**, **!**
- comparaciones: **<**, **>**, **<=**, **>=**, **!=**, **==**
- Acceso por índice en un arreglo estático: **arr[2]** (1op), **mat[0][3]** (2op), ...
- Acceso a la propiedad de un objeto: **arr.length**
- asignaciones: **a=15**, **str="mensaje"**, ...
- return de una función

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```


Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)
6. ||

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)
6. ||
7. acceso a índice (mat[0])

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)
6. ||
7. acceso a índice (mat[0])
8. .length (mat[0].length)

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)
6. ||
7. acceso a índice (mat[0])
8. .length (mat[0].length)
9. == (entre mat[0].length y 0)

Ejemplo 1

```
boolean estaVacía(int[][] mat) {  
    return mat==null || mat.length==0 || mat[0].length==0;  
}
```

1. return
2. == (entre mat y null)
3. ||
4. .length (mat.length)
5. == (entre mat.length y 0)
6. ||
7. acceso a índice (mat[0])
8. .length (mat[0].length)
9. == (entre mat[0].length y 0)

Complejidad: 9 operaciones

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

¿Cuál es la complejidad?

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

¿Cuál es la complejidad? ¿?

¿Es constante?

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

¿Cuál es la complejidad? ¿?

¿Es constante? **NO**

¿De qué depende?

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

¿Cuál es la complejidad? ¿?

¿Es constante? **NO**

¿De qué depende? → Depende del tamaño de la entrada

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

¿Cuál es la complejidad? ¿?

¿Es constante? **NO**

¿De qué depende? → Depende del tamaño de la entrada

La entrada es arr y el tamaño es la cantidad de elementos que tiene.

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true;  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo `arr` (`arr.length`)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares = true; // 1  
    for( int i = 0; i < arr.length; i++) // 1 + 2*(n+1) + n = 3+3n  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0;  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo `arr` (`arr.length`)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n = 3+3n  
        sonTodosPares = sonTodosPares && arr[i]%2 == 0; // 5  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n = 3+3n  
        | sonTodosPares = sonTodosPares && arr[i]%2 == 0; //[5]*n  
    return sonTodosPares;  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n = 3+3n  
        | sonTodosPares = sonTodosPares && arr[i]%2 == 0; //[5]*n  
    return sonTodosPares; // 1  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n = 3+3n  
        | sonTodosPares = sonTodosPares && arr[i]%2 == 0; //[5]*n  
    return sonTodosPares; // 1  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

$$f(n) = 1 + 3 + 3n + 5n + 1 = 8n + 5$$

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares=true; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1) + n = 3+3n  
        | sonTodosPares = sonTodosPares && arr[i]%2 == 0; //[5]*n  
    return sonTodosPares; // 1  
}
```

Definimos n = cantidad de elementos del arreglo arr (arr.length)

$$f(n) = 1 + 3 + 3n + 5n + 1 = 8n + 5$$

$$f(n) = 8n + 5$$

Ejemplo 2

```
boolean todosPares(int[] arr) {  
    boolean sonTodosPares = true; // 1  
    for( int i=0; i<arr.length; i++) // 1 + 2*(n+1) + n = 3+3n  
        | sonTodosPares = sonTodosPares && arr[i]%2 == 0; // [5]*n  
    return sonTodosPares; // 1  
}
```

Definimos n = cantidad de elementos del arreglo `arr` (`arr.length`)

$$f(n) = 1 + 3 + 3n + 5n + 1 = 8n + 5$$

$f(n) = 8n + 5 \rightarrow$ Esta **función de complejidad** nos permite calcular la cantidad de instrucciones de `todosPares(...)` para cualquier tamaño de entrada.

Función de complejidad

Es una función que estima la cantidad de instrucciones a ejecutarse para un algoritmo dependiendo del tamaño de la entrada.

Es muy útil para poder comparar dos soluciones a un mismo problema y elegir la de menor complejidad.

Es necesario definir una variable que represente la entrada del problema, como en el ejemplo2:

$f(n) = 8n + 5$, siendo n =cantidad de elementos del parámetro arr.

Análisis del peor caso

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0;  
    for( int i=0; i<arr.length;i++)  
        if (max < arr[i])  
            max = arr[i];  
    return max;  
}
```

Podemos interpretar que no siempre la condición del IF será positiva.

¿Cuál sería el peor caso? Realizar la cantidad máxima de instrucciones posible

En este caso, que siempre entre al cuerpo del IF.

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0;  
    for( int i=0; i<arr.length;i++)  
        if (max < arr[i])  
            max = arr[i];  
    return max;  
}
```

1ro definimos la variable a usar.

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0;  
    for( int i=0; i<arr.length;i++)  
        if (max < arr[i])  
            max = arr[i];  
    return max;  
}
```

1ro definimos la variable a usar.

n=cantidad de elementos de arr.

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0;  
    for( int i=0; i<arr.length;i++)  
        if (max < arr[i])  
            max = arr[i];  
    return max;  
}
```

1ro definimos la variable a usar.

n=cantidad de elementos de arr.

2do contamos línea a línea...

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)+ 1*n  
        if (max < arr[i]) //[ 2 ]*n  
            max = arr[i];    //[ 2 ]*n  
    return max; // 1  
}
```

1ro definimos la variable a usar.

n=cantidad de elementos de arr.

2do contamos línea a línea...

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)+ 1*n  
        if (max < arr[i]) // [ 2 ]*n  
            max = arr[i]; // [ 2 ]*n  
    return max; // 1  
}
```

n=cantidad de elementos de arr.

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)+ 1*n = 3+3n  
        if (max < arr[i]) //[ 2 ]*n  
            max = arr[i];    //[ 2 ]*n  
    return max; // 1  
}
```

n=cantidad de elementos de arr.

$$f(n) = 1 + 3 + 3n + 2n + 2n + 1 = 7n + 5$$

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)+ 1*n = 3+3n  
        if (max < arr[i]) //[ 2 ]*n  
            max = arr[i];    //[ 2 ]*n  
    return max; // 1  
}
```

n=cantidad de elementos de arr.

$f(n) = 1 + 3 + 3n + 2n + 2n + 1 = 7n + 5$ (En el peor caso)

Ejemplo 3

```
int maximo(int[] arr) {  
    int max=0; // 1  
    for( int i=0; i<arr.length;i++) // 1+ 2*(n+1)+ 1*n = 3+3n  
        if (max < arr[i]) //[ 2 ]*n  
            max = arr[i];    //[ 2 ]*n  
    return max; // 1  
}
```

n=cantidad de elementos de arr.

$f(n) = 1 + 3 + 3n + 2n + 2n + 1 = 7n + 5$ (En el peor caso)

Peor caso: recibir un arreglo ordenado de menor a mayor sin repetidos.

¿Consultas?